LEZIONE 6 - Primo Programma in C

Durata: 4 ore | Teoria: 2h | Esercizi: 2h

PARTE TEORICA (2 ore)

6.1 Introduzione al Linguaggio C (15 min)

Storia del C

Timeline:

- **1972**: Dennis Ritchie crea il C ai Bell Labs (AT&T)
- **1978**: Pubblicazione “The C Programming Language” (K&R)
- **1989**: Standard ANSI C (C89)
- **1999**: Standard C99 (nuove funzionalità)
- **2011**: Standard C11 (thread, atomics)
- **2018**: Standard C18 (correzioni bug)

Perché Studiare C?

Vantaggi:

- **Vicino all’hardware**: controllo diretto su memoria e CPU
- **Efficiente**: codice veloce e compatto
- **Portabile**: gira su qualsiasi piattaforma
- **Fondamentale**: base per C++, Java, C#, JavaScript
- **Diffuso**: sistemi operativi, embedded, database, compilatori

Utilizzi del C:

- Sistemi operativi (Linux, Windows kernel)
- Database (MySQL, PostgreSQL, SQLite)
- Linguaggi (Python, Ruby, PHP interpreters)
- Embedded systems (Arduino, IoT)

- Driver hardware
- Applicazioni real-time

Caratteristiche:

- Linguaggio di medio livello (tra assembly e alto livello)
- Compilato (non interpretato)
- Tipizzazione statica
- Gestione manuale della memoria
- Preprocessore potente

6.2 Ambiente di Sviluppo (10 min)

Strumenti Necessari

1. Compilatore:

- **GCC** (GNU Compiler Collection) - Linux, macOS, Windows
- **Clang** - LLVM, moderno e veloce
- **MSVC** (Microsoft Visual C++) - Windows
- **MinGW** - GCC per Windows

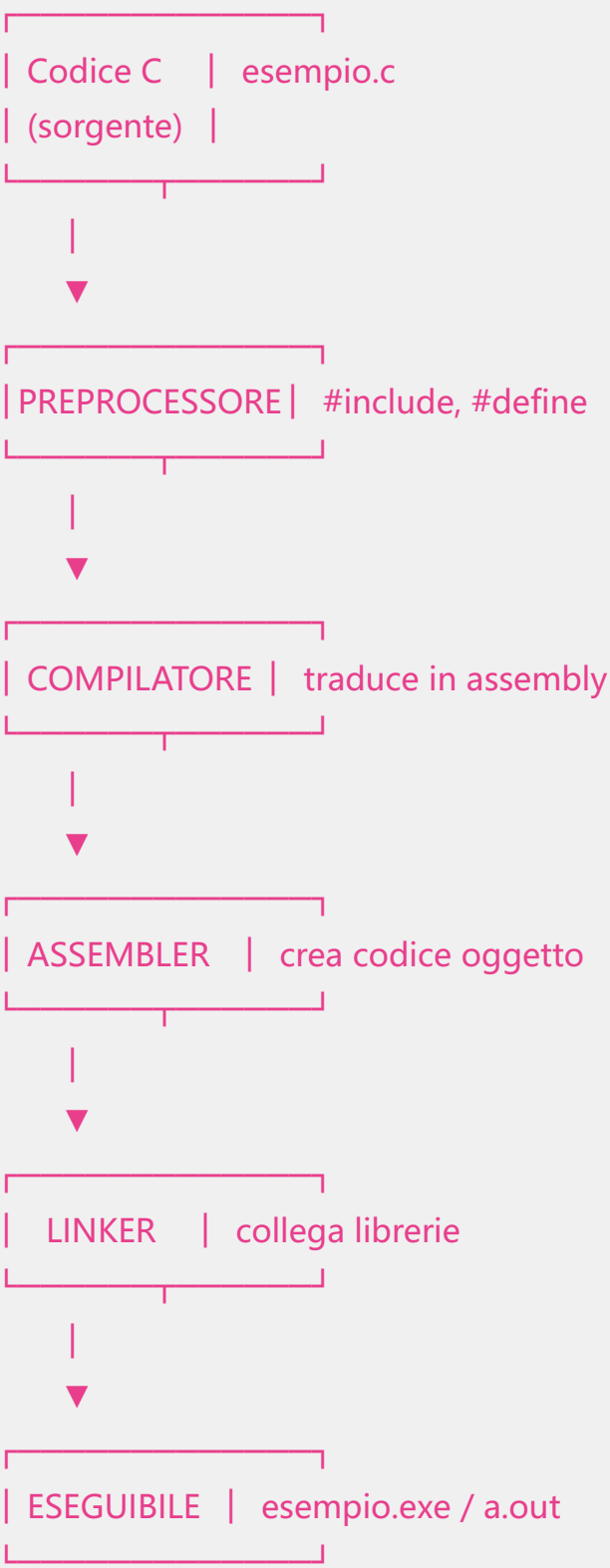
2. Editor/IDE:

- **Visual Studio Code**: leggero, estensioni
- **Code::Blocks**: IDE specifico per C/C++
- **CLion**: professionale (JetBrains)
- **Dev-C++**: semplice per principianti
- **Vim/Emacs**: per puristi

3. Debugger:

- **GDB** (GNU Debugger)
- **LLDB** (LLVM Debugger)
- Integrato nell'IDE

Processo di Compilazione



Comandi base:

```
# Compilazione semplice
gcc programma.c -o programma

# Con warning abilitati (RACCOMANDATO!)
gcc -Wall -Wextra programma.c -o programma

# Con informazioni di debug
gcc -g programma.c -o programma

# Ottimizzazione
gcc -O2 programma.c -o programma

# Solo preprocessing
gcc -E programma.c

# Solo compilazione (no linking)
gcc -c programma.c
```

6.3 Struttura di un Programma C (20 min)

Il Primo Programma: Hello World

```
#include <stdio.h>

int main() {
    printf("Numero: %d\n", 42)
    printf("Testo: %s\n" "Ciao");
    Return 0;
```

Errori:

1. Manca # prima di include
2. Manca ; dopo primo printf

3. Manca `,` tra formato e stringa nel secondo printf
4. `Return` dovrebbe essere minuscolo `return`
5. Manca `}` di chiusura

Codice corretto:

```c
#include <stdio.h>

int main() {
    printf("Numero: %d\n", 42);
    printf("Testo: %s\n", "Ciao");
    return 0;
}
```

4.3 Errori logici (compila ma risultato sbagliato):

```c
#include <stdio.h>

int main() {
    printf("Percentuale: %d%%\n", 50);  // Corretto: doppio %
    printf("Percorso: C:\nuovo\test\n"); // ERRORE: \n interpretato come
            newline!
    return 0;
}
```

Output errato:

```
Percentuale: 50%
Percorso: C:
uovo    est
```

Codice corretto:

```c
#include <stdio.h>
```

```c
int main() {
    printf("Percentuale: %d%%\n", 50);
    printf("Percorso: C:\\nuovo\\test\n"); // Backslash doppio!
    return 0;
}
```

Output corretto:

```
Percentuale: 50%
Percorso: C:\nuovo\test
```

Esercizio 5: Progetti Completi (45 min)

5.1 Biglietto di presentazione:

```c
/*
 * Programma: Biglietto da Visita
 * Crea un biglietto di presentazione formattato
 */

#include <stdio.h>

int main() {
    printf("╔══════════════════════════════════════╗\n");
    printf("║                                      ║\n");
    printf("║        MARIO ROSSI                   ║\n");
    printf("║        Programmatore C               ║\n");
    printf("║                                      ║\n");
    printf("║  Email:  mario.rossi@email.com       ║\n");
    printf("║  Tel:    +39 123 456 7890            ║\n");
    printf("║  Web:    www.mariorossi.it           ║\n");
    printf("║                                      ║\n");
    printf("╚══════════════════════════════════════╝\n");
```

```
    return 0;
}
```

5.2 Calcolatrice valori (solo output, no input ancora):

```
/*
 * Programma: Calcolatrice Semplice
 * Esegue operazioni su due numeri predefiniti
 */

#include <stdio.h>

int main() {
    // Numeri da calcolare
    int a = 15;
    int b = 4;

    printf("====== CALCOLATRICE ======\n");
    printf("Primo numero:  %d\n", a);
    printf("Secondo numero: %d\n", b);
    printf("==========================\n");

    // Operazioni
    printf("%d + %d = %d\n", a, b, a + b);
    printf("%d - %d = %d\n", a, b, a - b);
    printf("%d × %d = %d\n", a, b, a * b);
    printf("%d ÷ %d = %d (quoziente)\n", a, b, a / b);
    printf("%d %% %d = %d (resto)\n", a, b, a % b);

    printf("==========================\n");

    return 0;
}
```

Output:

```
====== CALCOLATRICE ======
Primo numero:  15
Secondo numero: 4
============================
15 + 4 = 19
15 - 4 = 11
15 × 4 = 60
15 ÷ 4 = 3 (quoziente)
15 % 4 = 3 (resto)
============================
```

5.3 Tabella ASCII:

```c
/*
 * Programma: Tabella ASCII
 * Stampa una porzione della tabella ASCII
 */

#include <stdio.h>

int main() {
    printf("=== TABELLA ASCII (caratteri stampabili) ===\n\n");
    printf("Dec  Hex  Char  |  Dec  Hex  Char\n");
    printf("---------------------------------\n");

    // Caratteri da 32 a 126 (stampabili)
    printf(" 32  0x20  '%c'  |  65  0x41  '%c'\n", 32, 65);
    printf(" 33  0x21  '%c'  |  66  0x42  '%c'\n", 33, 66);
    printf(" 34  0x22  '%c'  |  67  0x43  '%c'\n", 34, 67);
    printf(" 48  0x30  '%c'  |  97  0x61  '%c'\n", 48, 97);
    printf(" 49  0x31  '%c'  |  98  0x62  '%c'\n", 49, 98);
    printf(" 50  0x32  '%c'  |  99  0x63  '%c'\n", 50, 99);
```

```
    printf("\nNote:\n");
    printf("- 32 = spazio\n");
    printf("- 48-57 = cifre '0'-'9'\n");
    printf("- 65-90 = lettere 'A'-'Z'\n");
    printf("- 97-122 = lettere 'a'-'z'\n");

    return 0;
}
```

5.4 Conversione temperature (valori fissi):

```
/*
 * Programma: Convertitore Temperature
 * Converte da Celsius a Fahrenheit e Kelvin
 */

#include <stdio.h>

int main() {
    float celsius = 25.0;
    float fahrenheit, kelvin;

    // Conversioni
    fahrenheit = (celsius * 9.0 / 5.0) + 32.0;
    kelvin = celsius + 273.15;

    printf("===== CONVERSIONE TEMPERATURE =====\n");
    printf("Temperatura in Celsius:    %.2f°C\n", celsius);
    printf("---------------------------------\n");
    printf("In Fahrenheit:             %.2f°F\n", fahrenheit);
    printf("In Kelvin:                 %.2fK\n", kelvin);
    printf("=================================\n");
```

```
    return 0;
}
```

Output:

```
===== CONVERSIONE TEMPERATURE =====
Temperatura in Celsius:    25.00°C
-----------------------------------
In Fahrenheit:             77.00°F
In Kelvin:                 298.15K
=====================================
```

5.5 Menu informativo:

```
/*
 * Programma: Sistema Informativo
 * Mostra informazioni sul sistema
 */

#include <stdio.h>

int main() {
    printf("\n");
    printf(" ╔═══════════════════════════════════════════════════════╗\n");
    printf(" ║    SISTEMA INFORMATIVO - v1.0         ║\n");
    printf(" ╚═══════════════════════════════════════════════════════╝\n");
    printf("\n");

    printf(" ┌─────────────────────────────────────────────────────┐\n");
    printf(" │ INFORMAZIONI SUL PROGRAMMA            │\n");
    printf(" ├─────────────────────────────────────────────────────┤\n");
    printf(" │ Compilato con: GCC                    │\n");
    printf(" │ Standard:      C99                    │\n");
    printf(" │ Dimensione:    ~8 KB                  │\n");
```

```
    printf("| Architettura:  x86-64                |\n");
    printf("└──────────────────────────────────────────────────────────┘\n");
    printf("\n");

    printf("┌──────────────────────────────────────────────────────────┐\n");
    printf("| LIMITI TIPI DI DATO (tipici)         |\n");
    printf("├──────────────────────────────────────────────────────────┤\n");
    printf("| int:     %d byte                   |\n", (int)sizeof(int));
    printf("| float:   %d byte                   |\n", (int)sizeof(float));
    printf("| double:  %d byte                   |\n", (int)sizeof(double));
    printf("| char:    %d byte                   |\n", (int)sizeof(char));
    printf("└──────────────────────────────────────────────────────────┘\n");
    printf("\n");

    printf("Premi INVIO per terminare...\n");

    return 0;
}
```

Congratulazioni! Hai scritto i tuoi primi programmi in C! 🎉

Ora conosci la struttura base di un programma C e sai come stampare output formattato. Nelle prossime lezioni aggiungeremo input, variabili e logica per creare programmi interattivi e potenti!) {
printf(“Hello, World!\n”);
return 0;
}

Analisi riga per riga:

Riga 1: `#include <stdio.h>`

```c
#include <stdio.h>
```

- **#include**: direttiva del preprocessore
- **<stdio.h>**: header file (Standard Input/Output)
- Include funzioni per I/O: printf, scanf, getchar, etc.
- Il preprocessore "copia-incolla" il contenuto di stdio.h

Riga 3: int main() {

```c
int main() {
```

- **int**: tipo di ritorno della funzione (intero)
- **main**: nome della funzione principale
- **()**: lista parametri (vuota in questo caso)
- **{**: inizio blocco funzione
- main() è il **punto di ingresso** del programma

Riga 4: printf("Hello, World!\n");

```c
    printf("Hello, World!\n");
```

- **printf**: funzione per stampare (print formatted)
- **"Hello, World!\n"**: stringa da stampare
- **\n**: carattere newline (a capo)
- **;**: terminatore istruzione (obbligatorio!)

Riga 5: return 0;

```c
    return 0;
```

- **return**: restituisce un valore
- **0**: codice di uscita (0 = successo)
- Convenzione: 0 = OK, diverso da 0 = errore
- Il sistema operativo riceve questo codice

Riga 6: }

```
}
```

- }: fine blocco funzione

Struttura Generale

```
// 1. DIRETTIVE PREPROCESSORE
#include <libreria.h>
#define COSTANTE valore

// 2. DICHIARAZIONI GLOBALI
int variabile_globale;

// 3. PROTOTIPI FUNZIONI
void mia_funzione(int parametro);

// 4. FUNZIONE MAIN (obbligatoria)
int main() {
    // Istruzioni
    return 0;
}

// 5. DEFINIZIONI FUNZIONI
void mia_funzione(int parametro) {
    // Corpo funzione
}
```

Varianti di main()

Forma 1: Senza parametri

```c
int main() {
    return 0;
}
```

Forma 2: Con void esplicito

```c
int main(void) {
    return 0;
}
```

Forma 3: Con argomenti da riga di comando

```c
int main(int argc, char *argv[]) {
    // argc = numero argomenti
    // argv = array di stringhe (argomenti)
    return 0;
}
```

Forma 4: Con variabili ambiente (non standard)

```c
int main(int argc, char *argv[], char *envp[]) {
    // envp = variabili ambiente
    return 0;
}
```

6.4 La Funzione printf() (25 min)

Sintassi Base

```c
printf("formato", argomenti);
```

Esempi:

```c
printf("Ciao");            // Stampa stringa semplice
printf("Numero: %d", 42);    // Stampa numero
printf("X=%d, Y=%d", 10, 20); // Stampa due numeri
```

Specificatori di Formato

Gli **specificatori** iniziano con `%` e indicano come stampare i dati.

Tabella completa:

Specificatore	Tipo	Esempio	Output
`%d`	int (decimale)	`printf("%d", 42)`	42
`%i`	int (decimale)	`printf("%i", 42)`	42
`%u`	unsigned int	`printf("%u", 42)`	42
`%ld`	long	`printf("%ld", 123456L)`	123456
`%lld`	long long	`printf("%lld", 123LL)`	123
`%f`	float/double	`printf("%f", 3.14)`	3.140000
`%.2f`	float (2 decimali)	`printf("%.2f", 3.14)`	3.14
`%e`	notazione scientifica	`printf("%e", 1234.5)`	1.234500e+03
`%c`	char (carattere)	`printf("%c", 'A')`	A
`%s`	stringa	`printf("%s", "ciao")`	ciao
`%p`	puntatore	`printf("%p", ptr)`	0x7fff5fbff6a0
`%x`	esadecimale minuscolo	`printf("%x", 255)`	ff
`%X`		`printf("%X", 255)`	FF

Specificatore	Tipo	Esempio	Output
	esadecimale MAIUSCOLO		
%o	ottale	printf("%o", 8)	10
%%	carattere %	printf("100%%")	100%

Modificatori di Larghezza e Precisione

Larghezza minima:

```
printf("%5d", 42);      // "   42" (5 caratteri, allineato a destra)
printf("%-5d", 42);     // "42   " (allineato a sinistra)
printf("%05d", 42);     // "00042" (riempito con zeri)
```

Precisione per float:

```
printf("%.2f", 3.14159);   // "3.14" (2 decimali)
printf("%.4f", 3.14159);   // "3.1416" (4 decimali, arrotondato)
printf("%8.2f", 3.14);     // "    3.14" (larghezza 8, 2 decimali)
```

Precisione per stringhe:

```
printf("%.5s", "Hello World");  // "Hello" (primi 5 caratteri)
```

Sequenze di Escape

Le **sequenze di escape** iniziano con \ e rappresentano caratteri speciali.

Sequenza	Significato	Codice ASCII
\n	Newline (a capo)	10
\t	Tab orizzontale	9

Sequenza	Significato	Codice ASCII
\\	Backslash	92
\'	Apice singolo	39
\"	Doppi apici	34
\0	Null (terminatore stringa)	0
\r	Carriage return	13
\b	Backspace	8
\a	Bell (suono)	7
\v	Tab verticale	11
\?	Punto interrogativo	63

Esempi:

```
printf("Riga 1\nRiga 2\n");       // Due righe
printf("Nome\tCognome\n");        // Separati da tab
printf("Percorso: C:\\Windows\n"); // Backslash
printf("Disse: \"Ciao\"\n");      // Virgolette
```

Esempi Completi

Esempio 1: Stampa formattata

```
#include <stdio.h>

int main() {
    int eta = 25;
    float altezza = 1.75;
    char iniziale = 'M';
```

```
    printf("Età: %d anni\n", eta);
    printf("Altezza: %.2f metri\n", altezza);
    printf("Iniziale: %c\n", iniziale);

    return 0;
}
```

Output:

```
Età: 25 anni
Altezza: 1.75 metri
Iniziale: M
```

Esempio 2: Tabella formattata

```
#include <stdio.h>

int main() {
    printf("%-10s %5s %8s\n", "Nome", "Età", "Stipendio");
    printf("%-10s %5d %8.2f\n", "Mario", 30, 1500.50);
    printf("%-10s %5d %8.2f\n", "Lucia", 28, 1800.75);
    printf("%-10s %5d %8.2f\n", "Giuseppe", 35, 2200.00);

    return 0;
}
```

Output:

```
Nome          Età Stipendio
Mario          30  1500.50
Lucia          28  1800.75
Giuseppe       35  2200.00
```

Esempio 3: Conversioni di base

```c
#include <stdio.h>

int main() {
    int numero = 255;

    printf("Decimale: %d\n", numero);
    printf("Esadecimale: %x\n", numero);
    printf("Esadecimale (maiuscolo): %X\n", numero);
    printf("Ottale: %o\n", numero);
    printf("Binario: (non supportato direttamente)\n");

    return 0;
}
```

Output:

```
Decimale: 255
Esadecimale: ff
Esadecimale (maiuscolo): FF
Ottale: 377
Binario: (non supportato direttamente)
```

6.5 Commenti nel Codice (15 min)

I **commenti** sono testo ignorato dal compilatore, usati per documentare il codice.

Commenti su Singola Riga

```c
// Questo è un commento su una riga

int x = 5; // Commento dopo codice
```

Commenti Multi-Riga

```
/*
 * Questo è un commento
 * su più righe
 */

/* Anche questo funziona */
```

Stili di Documentazione

Stile 1: Breve e conciso

```
// Calcola la media di due numeri
float media = (a + b) / 2.0;
```

Stile 2: Documentazione funzione

```
/*
 * Calcola il fattoriale di un numero
 *
 * Parametri:
 *   n - numero intero positivo
 *
 * Ritorna:
 *   n! (fattoriale di n)
 *
 * Nota: non gestisce overflow
 */
int fattoriale(int n) {
    // Implementazione...
}
```

Stile 3: Doxygen (documentazione automatica)

```
/**
 * @brief Calcola la somma di due numeri
 *
 * @param a Primo numero
 * @param b Secondo numero
 * @return Somma di a e b
 */
int somma(int a, int b) {
    return a + b;
}
```

Best Practices per i Commenti

✓ BUONI commenti:

```
// Converti temperatura da Celsius a Fahrenheit
float fahrenheit = celsius * 9.0 / 5.0 + 32.0;

// FIXME: Questo algoritmo è O(n²), da ottimizzare
void ordinamento_lento(int array[], int n) {
    // ...
}

// TODO: Aggiungere gestione errori
int dividi(int a, int b) {
    return a / b;
}
```

✗ CATTIVI commenti:

```
// Incrementa i
i++; // Ovvio!

// Questa funzione somma a e b
```

```c
int somma(int a, int b) { // Ridondante, il nome è chiaro
    return a + b;
}

// Codice scritto il 15/03/2023 da Mario
// Non toccare!!! (Cattiva pratica)
```

Regole d'oro:

1. **Spiega il PERCHÉ, non il COSA**: il codice dice già cosa fa
2. **Mantieni aggiornati**: commenti obsoleti sono peggio di nessun commento
3. **Sii conciso**: commenti troppo lunghi non vengono letti
4. **Usa nomi variabili chiari**: riduce bisogno di commenti
5. **Documenta algoritmi complessi**: aiuta chi legge (incluso te futuro)

6.6 Compilazione ed Esecuzione (15 min)

Step-by-Step: Dal Codice all'Esecuzione

1. Scrivi il codice

```c
// hello.c
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

2. Salva il file

- Nome: `hello.c` (estensione `.c` obbligatoria)
- Usa editor di testo semplice (non Word!)

3. Compila

```
gcc hello.c -o hello
```

- `gcc`: compilatore
- `hello.c`: file sorgente
- `-o hello`: output eseguibile (nome “hello”)

4. Esegui

```
# Linux/macOS
./hello

# Windows
hello.exe
```

5. Vedi output

```
Hello, World!
```

Opzioni Utili del Compilatore

Warning (SEMPRE usare!):

```
gcc -Wall -Wextra -Werror hello.c -o hello
```

- `-Wall`: abilita warning comuni
- `-Wextra`: abilita warning extra
- `-Werror`: tratta warning come errori

Debug:

```
gcc -g hello.c -o hello
```

- `-g`: include simboli debug (per gdb)

Ottimizzazione:

```
gcc -O2 hello.c -o hello
```

- `-O0`: nessuna ottimizzazione (default)
- `-O1`: ottimizzazione base
- `-O2`: ottimizzazione media (raccomandato)
- `-O3`: ottimizzazione aggressiva
- `-Os`: ottimizza dimensione

Standard C:

```
gcc -std=c99 hello.c -o hello
```

- `-std=c89`: ANSI C (1989)
- `-std=c99`: C99 (1999)
- `-std=c11`: C11 (2011)
- `-std=c18`: C18 (2018)

Esempio completo (raccomandato per sviluppo):

```
gcc -Wall -Wextra -std=c99 -g hello.c -o hello
```

Errori Comuni di Compilazione

Errore 1: File non trovato

```
gcc ciao.c -o ciao
```

```
ciao.c: No such file or directory
```

Soluzione: Verifica nome file e percorso

Errore 2: Punto e virgola mancante

```
printf("Hello")  // Manca ;
```

```
error: expected ';' before 'return'
```

Soluzione: Aggiungi `;`

Errore 3: Include mancante

```c
// Manca #include <stdio.h>
int main() {
    printf("Hello");
}
```

```
warning: implicit declaration of function 'printf'
```

Soluzione: Aggiungi `#include <stdio.h>`

Errore 4: Parentesi non bilanciate

```c
int main() {
    printf("Hello\n");
    return 0;
// Manca }
```

```
error: expected declaration or statement at end of input
```

Soluzione: Chiudi tutte le parentesi

6.7 Stile e Indentazione (10 min)

Lo **stile** del codice è importante per leggibilità e manutenibilità.

Indentazione

Stile K&R (Kernighan & Ritchie):

```c
int main() {
    if (condizione) {
        istruzione1;
        istruzione2;
    } else {
        istruzione3;
    }
    return 0;
}
```

Stile Allman:

```c
int main()
{
    if (condizione)
    {
        istruzione1;
        istruzione2;
    }
    else
    {
        istruzione3;
    }
    return 0;
}
```

Raccomandazione: Scegli uno stile e mantienilo consistente!

Indentazione:

- **4 spazi** (raccomandato per C)
- **2 spazi** (compatto)
- **Tab** (evitare, problemi di visualizzazione)

Nomi Variabili

Convenzioni:

```c
// snake_case (raccomandato per C)
int numero_studenti;
float valore_medio;

// camelCase (comune in altri linguaggi)
int numeroStudenti;
float valoreMedio;

// PascalCase (per tipi/strutture)
struct PersonaDati {
    // ...
};
```

Regole:

- Nomi descrittivi: `temperatura` non `t`
- Costanti in MAIUSCOLO: `MAX_SIZE`, `PI`
- Evita abbreviazioni criptiche
- Non usare parole chiave riservate

Spazi e Formattazione

Operatori:

```c
// ✓ BUONO
x = a + b * c;
if (x > 0) { }

// ✗ CATTIVO
x=a+b*c;
if(x>0){ }
```

Funzioni:

```
// ✓ BUONO
printf("Hello");
somma(a, b);

// ✗ CATTIVO
printf ("Hello");
somma (a,b);
```

PARTE PRATICA (2 ore)

ESERCIZI GUIDATI

Esercizio 1: Hello World e Varianti (20 min)

1.1 Scrivi e compila il classico “Hello, World!”:

```
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

Compila ed esegui:

```
gcc hello.c -o hello
./hello
```

Output atteso:

```
Hello, World!
```

1.2 Modifica per stampare il tuo nome:

```c
#include <stdio.h>

int main() {
    printf("Ciao, mi chiamo Mario!\n");
    return 0;
}
```

1.3 Stampa su più righe:

```c
#include <stdio.h>

int main() {
    printf("Benvenuto nel corso di C!\n");
    printf("Questa è la lezione 6.\n");
    printf("Buono studio!\n");
    return 0;
}
```

Output:

```
Benvenuto nel corso di C!
Questa è la lezione 6.
Buono studio!
```

1.4 Usa un'unica printf con sequenze di escape:

```c
#include <stdio.h>

int main() {
```

```
    printf("Riga 1\nRiga 2\nRiga 3\n");
    return 0;
}
```

1.5 Crea un “disegno” ASCII:

```
#include <stdio.h>

int main() {
    printf("  *****  \n");
    printf(" *     * \n");
    printf("*  O O  *\n");
    printf("*   ^   *\n");
    printf(" * \_/ * \n");
    printf("  *****  \n");
    return 0;
}
```

Output:

```
 *****
*     *
*  O O  *
*   ^   *
 * \_/ *
  *****
```

Esercizio 2: Esperimenti con printf() (25 min)

2.1 Stampa numeri interi:

```
#include <stdio.h>
```

```c
int main() {
    printf("Numero: %d\n", 42);
    printf("Negativo: %d\n", -15);
    printf("Grande: %d\n", 1000000);

    return 0;
}
```

Output:

```
Numero: 42
Negativo: -15
Grande: 1000000
```

2.2 Stampa numeri decimali:

```c
#include <stdio.h>

int main() {
    printf("Pi greco: %f\n", 3.14159);
    printf("Pi (2 decimali): %.2f\n", 3.14159);
    printf("Piccolo: %.10f\n", 0.0000001);
    printf("Scientifico: %e\n", 1234567.89);

    return 0;
}
```

Output:

```
Pi greco: 3.141590
Pi (2 decimali): 3.14
Piccolo: 0.0000001000
Scientifico: 1.234568e+06
```

2.3 Stampa caratteri e stringhe:

```c
#include <stdio.h>

int main() {
    printf("Carattere: %c\n", 'A');
    printf("Codice ASCII: %d\n", 'A');
    printf("Stringa: %s\n", "Programmazione");
    printf("Primi 5 caratteri: %.5s\n", "Programmazione");

    return 0;
}
```

Output:

```
Carattere: A
Codice ASCII: 65
Stringa: Programmazione
Primi 5 caratteri: Progr
```

2.4 Formato tabellare:

```c
#include <stdio.h>

int main() {
    printf("%-15s %10s %8s\n", "Prodotto", "Quantità", "Prezzo");
    printf("%-15s %10d %8.2f€\n", "Mele", 10, 2.50);
    printf("%-15s %10d %8.2f€\n", "Arance", 5, 3.20);
    printf("%-15s %10d %8.2f€\n", "Banane", 12, 1.80);

    return 0;
}
```

Output:

```
Prodotto        Quantità   Prezzo
Mele               10     2.50€
Arance              5     3.20€
Banane             12     1.80€
```

2.5 Conversioni di base:

```c
#include <stdio.h>

int main() {
    int num = 255;

    printf("Decimale:     %d\n", num);
    printf("Esadecimale:  0x%X\n", num);
    printf("Ottale:       0%o\n", num);
    printf("Carattere:    '%c'\n", num);

    return 0;
}
```

Output:

```
Decimale:     255
Esadecimale:  0xFF
Ottale:       0377
Carattere:    'ÿ'
```

Esercizio 3: Commenti e Documentazione (20 min)

3.1 Documenta un programma semplice:

```c
/*
 * Programma: Calcolatore età
```

```
 * Autore: Mario Rossi
 * Data: 10/10/2024
 * Descrizione: Stampa l'anno di nascita data l'età
 */

#include <stdio.h>

int main() {
    // Anno corrente
    int anno_corrente = 2024;

    // Età della persona
    int eta = 25;

    // Calcola anno di nascita (approssimativo)
    int anno_nascita = anno_corrente - eta;

    // Stampa risultato
    printf("Se hai %d anni, sei nato nel %d\n", eta, anno_nascita);

    return 0; // Successo
}
```

3.2 Commenta codice esistente:

```
#include <stdio.h>

int main() {
    // Prezzi dei prodotti (in euro)
    float prezzo_mela = 0.50;
    float prezzo_pane = 1.20;
    float prezzo_latte = 0.95;

    // Quantità acquistate
```

```c
    int qta_mele = 5;
    int qta_pane = 2;
    int qta_latte = 3;

    // Calcolo totale spesa
    float totale = (prezzo_mela * qta_mele) +
                   (prezzo_pane * qta_pane) +
                   (prezzo_latte * qta_latte);

    // Visualizza scontrino
    printf("=== SCONTRINO ===\n");
    printf("Mele:  %d × %.2f€ = %.2f€\n", qta_mele, prezzo_mela, prezzo_mela
           * qta_mele);
    printf("Pane:  %d × %.2f€ = %.2f€\n", qta_pane, prezzo_pane, prezzo_pane
           * qta_pane);
    printf("Latte: %d × %.2f€ = %.2f€\n", qta_latte, prezzo_latte, prezzo_latte *
           qta_latte);
    printf("==================\n");
    printf("TOTALE: %.2f€\n", totale);

    return 0;
}
```

Esercizio 4: Debug Errori Comuni (30 min)

4.1 Trova e correggi gli errori:

Codice con errori:

```c
#include <stdio.h>

int main() {
    printf("Hello World)
    return 0
}
```

Errori:

1. Manca " di chiusura in printf
2. Manca ; dopo printf
3. Manca ; dopo return

Codice corretto:

```c
#include <stdio.h>

int main() {
    printf("Hello World\n");
    return 0;
}
```

4.2 Codice con errori multipli:

ESERCIZI AUTONOMI

Esercizio 6: Creatività

6.1 Crea un programma che stampa un banner ASCII con il tuo nome.

6.2 Crea una “ricevuta fiscale” formattata con almeno 5 prodotti.

6.3 Crea un programma che mostra la tua “schedina” sportiva preferita (squadre e punteggi).

6.4 Disegna una bandiera usando caratteri ASCII.

6.5 Crea un “orologio” che mostra l’ora (fissa, es. 14:30:45) in formato digitale ASCII.

ESERCIZI DA SVOLGERE A CASA

Homework 1: Scrivi un programma che stampa la tua presentazione personale includendo:

- Nome completo
- Età e data di nascita
- Città di residenza
- 3 hobby preferiti
- Messaggio di chiusura

Usa printf formattato e sequenze di escape per renderlo leggibile.

Homework 2: Crea un programma che converte:

- Un valore in metri in: centimetri, millimetri, chilometri
- Stampa i risultati in formato tabellare

Homework 3: Scrivi un programma che calcola e visualizza:

- Perimetro e area di un rettangolo (dati base e altezza)
- Perimetro e area di un cerchio (dato raggio)
- Usa π = 3.14159

Homework 4: Crea un “menu di ristorante” formattato con:

- Intestazione
- Antipasti (almeno 3) con prezzi
- Primi (almeno 3) con prezzi
- Secondi (almeno 3) con prezzi
- Dolci (almeno 2) con prezzi
- Usa allineamento e formattazione

Homework 5: Ricerca ed esperimenta:

- Cosa fa la funzione `puts()`?
- Differenza tra `printf()` e `puts()`
- Cosa fa `putchar()`?
- Scrivi esempi pratici

Homework 6: Crea un programma che mostra una tabella di conversione:

- Numeri da 0 a 16
- In decimale, binario (scritto manualmente), ottale, esadecimale
- Formato tabellare pulito

Esempio output:

```
Dec  Bin   Oct  Hex
 0  0000   0    0
 1  0001   1    1
 2  0010   2    2
...
 16 10000  20   10
```

APPROFONDIMENTI

Storia del “Hello, World!”

Origine:

- Prima apparizione: “A Tutorial Introduction to the Language B” (Brian Kernighan, 1972)
- Reso famoso da “The C Programming Language” (K&R, 1978)
- Diventato lo standard de facto per test di linguaggi

Versione originale (B language):

```
main() {
    extrn a, b, c;
    putchar(a); putchar(b); putchar(c);
}
a 'hel';
b 'lo,';
c ' wo\n';
```

Versione C (K&R 1978):

```
main()
{
    printf("hello, world\n");
}
```

Versione moderna:

```
#include <stdio.h>

int main(void) {
    printf("Hello, World!\n");
    return 0;
}
```

Printf: Storia e Funzionamento Interno

Origine nome: “print formatted” (stampa formattata)

Implementazione semplificata:

```
// printf è una funzione variadica (numero variabile argomenti)
int printf(const char *format, ...) {
    // 1. Parse stringa formato
    // 2. Per ogni %X trovato:
    //    - Preleva prossimo argomento
    //    - Converti secondo specificatore
    //    - Stampa
    // 3. Ritorna numero caratteri stampati
}
```

Vulnerabilità “format string”:

```
// PERICOLOSO! Mai fare così:
char input[100];
gets(input); // Input da utente
printf(input); // BUG DI SICUREZZA!

// Se input = "%x%x%x%x" legge memoria stack!

// SICURO:
printf("%s", input);
```

Preprocessore C

Il **preprocessore** elabora il codice PRIMA della compilazione.

Direttive principali:

1. #include:

```
#include <stdio.h>     // Header di sistema (cerchi standard)
#include "miofile.h"   // Header locale (directory corrente)
```

2. #define:

```
#define PI 3.14159
#define MAX(a,b) ((a) > (b) ? (a) : (b))

// Uso:
float area = PI * raggio * raggio;
int maggiore = MAX(10, 20);
```

3. Compilazione condizionale:

```
#ifdef DEBUG
    printf("Debug: x = %d\n", x);
```

```
#endif

#if VERSIONE >= 2
    nuova_funzione();
#else
    vecchia_funzione();
#endif
```

4. Pragma:

```
#pragma once  // Include una sola volta (alternativa a include guard)
```

Esempio espansione:

```
// Codice originale
#define QUADRATO(x) ((x) * (x))
int y = QUADRATO(5);

// Dopo preprocessore
int y = ((5) * (5));
```

Ottimizzazione Compilatore

Il compilatore può **ottimizzare** il codice per renderlo più veloce.

Esempio:

```
// Codice originale
int calcola() {
    int x = 2;
    int y = 3;
    int z = x * y + 1;
    return z;
}
```

```
// Dopo ottimizzazione (-O2)
int calcola() {
    return 7;  // Tutto calcolato a compile-time!
}
```

Livelli ottimizzazione:

- `-O0`: Nessuna (default, veloce compilazione)
- `-O1`: Base (poco impatto compile time)
- `-O2`: Media (raccomandato per release)
- `-O3`: Aggressiva (può aumentare dimensione)
- `-Os`: Dimensione (binario piccolo)
- `-Ofast`: Massima velocità (può violare standard)

Esempio impatto:

```
// Benchmark: somma 1 miliardo di numeri
int somma = 0;
for (int i = 0; i < 1000000000; i++) {
    somma += i;
}

Risultati:
-O0: 2.5 secondi
-O1: 1.2 secondi
-O2: 0.3 secondi
-O3: 0.001 secondi (loop eliminato! calcolo diretto)
```

Cross-Compilation

Puoi compilare per **piattaforme diverse** dalla tua.

Esempio: compilare per ARM su x86:

```
# Installa cross-compiler
sudo apt-get install gcc-arm-linux-gnueabi

# Compila per ARM
arm-linux-gnueabi-gcc -o hello_arm hello.c

# Non puoi eseguire direttamente su x86!
# Serve emulatore o dispositivo ARM
```

Usi:

- Sviluppo embedded (Arduino, Raspberry Pi)
- Sviluppo mobile
- Sistemi industriali

Standard Library vs System Calls

Standard Library (libc):

- Funzioni portabili (printf, malloc, fopen)
- Astratte dal sistema operativo
- Garantite su tutte le piattaforme

System Calls:

- Funzioni specifiche OS (Linux: fork, Windows: CreateProcess)
- Accesso diretto al kernel
- Non portabili

Esempio:

```
// Standard Library (portabile)
FILE *f = fopen("file.txt", "r");
fclose(f);

// System Call Linux (non portabile)
```

```
int fd = open("file.txt", O_RDONLY);
close(fd);
```

Undefined Behavior (UB)

Alcuni errori causano **comportamento indefinito**: il programma può fare QUALSIASI COSA.

Esempi UB:

```
// 1. Array out of bounds
int arr[5];
arr[10] = 42;  // UB!

// 2. Divisione per zero
int x = 10 / 0;  // UB!

// 3. Signed integer overflow
int max = 2147483647;
int y = max + 1;  // UB!

// 4. Dereferenziare NULL
int *p = NULL;
*p = 5;  // UB!

// 5. Usare variabile non inizializzata
int x;
printf("%d", x);  // UB!
```

Conseguenze UB:

- Crash
- Risultati casuali
- Funziona su PC ma non su server
- “Nasal demons” (folklore: può far uscire demoni dal naso!)

Prevenzione:

- Usa `-Wall -Wextra -Werror`
- Usa sanitizer: `-fsanitize=address,undefined`
- Inizializza sempre variabili
- Controlla limiti array

RIEPILOGO CONCETTI CHIAVE

Struttura Programma C

```
#include <librerie.h>  // Preprocessore

int main() {          // Funzione principale
    // Istruzioni
    return 0;         // Codice uscita
}
```

Printf Essenziale

```
printf("formato", argomenti);

Specificatori principali:
%d / %i  - int
%f       - float/double
%c       - char
%s       - stringa
%x / %X  - esadecimale
%p       - puntatore
%%       - carattere %
```

Sequenze Escape

```
\n  - newline
\t  - tab
\\  - backslash
\"  - doppi apici
\'  - apice singolo
```

Compilazione

```
gcc -Wall -Wextra file.c -o programma
./programma
```

TABELLE DI RIFERIMENTO RAPIDO

Specificatori Printf Comuni

Tipo	Specificatore	Esempio
int	%d	printf(“%d”, 42)
unsigned	%u	printf(“%u”, 42)
long	%ld	printf(“%ld”, 123456L)
float/double	%f	printf(“%f”, 3.14)
float (2 dec)	%.2f	printf(“%.2f”, 3.14)
char	%c	printf(“%c”, ‘A’)
string	%s	printf(“%s”, “ciao”)
hex	%x	printf(“%x”, 255)

Tipo	Specificatore	Esempio
pointer	%p	printf(“%p”, &var)

Opzioni GCC Utili

Opzione	Scopo
`-Wall`	Warning comuni
`-Wextra`	Warning extra
`-Werror`	Warning → errori
`-g`	Debug info
`-O2`	Ottimizzazione
`-std=c99`	Standard C99
`-o file`	Nome output
`-c`	Compila senza link

Caratteri Escape

Sequenza	Nome	ASCII
`\n`	Newline	10
`\t`	Tab	9
`\r`	Carriage return	13
`\\`	Backslash	92
`\'`	Apice	39
`\"`	Doppi apici	34

Sequenza	Nome	ASCII
\0	Null	0
\a	Bell	7

PROBLEMI COMUNI E SOLUZIONI

Problema 1: Printf non stampa

```
printf("Hello");  // Non appare!
```

Causa: Buffer non svuotato

Soluzione:

```
printf("Hello\n");  // \n svuota buffer
// oppure
printf("Hello");
fflush(stdout);
```

Problema 2: Warning “implicit declaration”

```
warning: implicit declaration of function 'printf'
```

Causa: Manca include

Soluzione:

```
#include <stdio.h>  // In cima al file
```

Problema 3: Caratteri strani su Windows

Causa: Encoding diverso (Windows usa CP-1252, non UTF-8)

Soluzione:

```
// Aggiungi in main():
#ifdef _WIN32
    system("chcp 65001 > nul");  // UTF-8 su Windows
#endif
```

Problema 4: Return type mismatch

```
warning: control reaches end of non-void function
```

Causa: Manca return in funzione

Soluzione:

```
int main() {
    printf("Hello\n");
    return 0;  // ← Obbligatorio!
}
```

PUNTI CHIAVE DELLA LEZIONE

✓ **main()** è il punto di ingresso, obbligatorio
✓ **#include <stdio.h>** per usare printf
✓ **printf()** stampa output formattato con specificatori %
✓ **return 0** indica successo (0 = OK, altro = errore)
✓ **;** termina ogni istruzione
✓ **Commenti** con `//` (singola) o `/* */` (multi-riga)
✓ **Compilazione**: `gcc file.c -o programma`

✓ **Indentazione** e stile: mantieni consistenza
✓ **Warning**: sempre abilitare con `-Wall -Wextra`
✓ **Documentazione**: commenta il PERCHÉ, non il COSA

PREPARAZIONE PER LA PROSSIMA LEZIONE

Nella prossima lezione studieremo:

- **Variabili e tipi di dato**: int, float, char, double
- **Dichiarazione e inizializzazione**
- **Input con scanf()**: leggere dati da utente
- **Operatori aritmetici**: +, -, *, /, %
- **Conversioni di tipo**: casting

Porta con te:

- Ambiente C funzionante
- Esercizi homework completati
- Domande su printf o compilazione
- Esperimenti fatti autonomamente

Verifica preparazione:

- Riesci a compilare ed eseguire programmi?
- Conosci i principali specificatori printf?
- Sai come commentare il codice?
- Hai familiarità con errori compilazione?

GLOSSARIO TECNICO

- **Compilatore**: Traduce codice C in eseguibile
- **Preprocessore**: Elabora direttive # prima compilazione
- **Header file**: File .h con dichiarazioni funzioni
- **main()**: Funzione principale, punto ingresso
- **printf()**: Funzione stampa formattata

- **Specificatore formato**: %d, %f, %s in printf
- **Sequenza escape**: \n, \t, \ caratteri speciali
- **Commento**: Testo ignorato dal compilatore
- **Indentazione**: Spaziatura per leggibilità
- **Warning**: Avviso compilatore (non errore fatale)
- **Linking**: Collegamento librerie a programma
- **Standard library**: Libreria funzioni C standard (libc)
- **Return code**: Valore ritornato da main (0 = successo)
- **Buffer**: Memoria temporanea per I/O
- **Undefined Behavior**: Comportamento non specificato standard

---